DIGITAL LOGIC DESIGN (EE1005) - MASTER FINAL EXAM REVISION GUIDE

FAST NUCES | Exam Weightage: Mid1 (15%) + Mid2 (20-25%) + Post-Mid2 (60-65%)

📊 CONTENT DISTRIBUTION BY WEIGHTAGE

Period	Weightage	Topics	Pages
Mid 1	15%	Number Systems, Boolean Algebra, K-Maps, Gates	§1-2
Mid 2	20-25%	Combinational Logic (MUX, Decoder), Adders	§3-4
Post-Mid 2	60-65%	Sequential Logic, Latches, Flip-Flops, Counters, Shift Registers	§5-8

SECTION 1: MID 1 TOPICS (15% Weightage)

1.1 NUMBER SYSTEMS & CONVERSIONS

Binary-Decimal Conversions

- **Decimal to Binary**: Divide by 2 repeatedly, collect remainders
- **Binary to Decimal**: Sum of (bit × 2^position)
- **Hex to Binary**: Each hex digit = 4 binary bits
- **Gray Code**: MSB same as binary MSB; subsequent bits = XOR of previous binary bits
  - Gray code advantage: Only 1 bit changes per transition (eliminates race conditions)

Signed Number Representation

Method	Range (4-bit)	Example (−5)	Key Use
Sign-Magnitude	−7 to +7	1101	Historical
1's Complement	−7 to +7	1010	Rarely used
2's Complement	−8 to +7	1011	**Standard**

2's Complement Calculation: Flip all bits, add 1.

1.2 BOOLEAN ALGEBRA & LOGIC GATES

Core Theorems (DeMorgan's Law Critical)

- **DeMorgan's 1st**: (A·B)' = A' + B'
- **DeMorgan's 2nd**: (A+B)' = A'·B'
- **Involution**: (A')' = A
- **Absorption**: A + A·B = A; A·(A+B) = A
- **Distributive**: A·(B+C) = A·B + A·C

Truth Tables for Critical Gates

Input	NOT	AND	OR	XOR	NAND	NOR	XNOR
0,0	—	0	0	0	1	1	1
0,1	—	0	1	1	1	0	0
1,0	—	0	1	1	1	0	0
1,1	—	1	1	0	0	0	1

Universal Gates: NAND, NOR can implement any logic.

1.3 KARNAUGH MAP (K-MAP) - ESSENTIAL FOR 10-15 MARKS

K-Map Minimization Rules (HIGH YIELD)

1. **Group adjacent 1s** in powers of 2 (2, 4, 8, 16 cells)
2. **Largest groups first** → fewer literals in final expression
3. **Don't cares (X)** → include if they reduce group count
4. **Wrap-around allowed** → edges and corners are adjacent
5. **Output 0**: Group 0s for complemented form, then apply DeMorgan's

4-Variable K-Map Layout (Most Common in Exams)

```
        AB
CD   00   01   11   10
00    0    1    2    3
01    4    5    6    7
11   12   13   14   15
10    8    9   10   11
```

Adjacency Rule: Row labels must differ by 1 bit; column labels must differ by 1 bit.

K-Map Pattern Recognition (Exam Strategy)

- **Single 1 in corner**: Must be included in pair or quad to minimize
- **Checkerboard pattern**: Cannot simplify beyond individual minterms
- **All 1s**: Output = 1 (regardless of inputs)
- **All 0s**: Output = 0

1.4 COMBINATIONAL CIRCUIT DESIGN METHODOLOGY

Step-by-Step Process

1. **Write truth table** with inputs and outputs
2. **Extract minterms** (rows where output = 1)
3. **Draw K-Map** and group 1s
4. **Read simplified expression** from groups
5. **Implement using gates** (AND-OR, NAND, or NOR)
6. **Verify** against original truth table

Example: 3-Input Majority Function (Exam Pattern)

- Output = 1 if at least 2 inputs = 1
- Truth table: m(3,5,6,7)
- K-Map groups: Two 4-cell groups (AB + AC + BC)
- Circuit: 3 AND gates → 1 OR gate

SECTION 2: MID 2 TOPICS (20-25% Weightage)

2.1 MULTIPLEXERS (MUX) & DECODERS

4-to-1 Multiplexer (Basic Building Block)

```
Truth Table: Select S[1:0] determines output
  S1 S0 | Output Y
   0  0 | I0
   0  1 | I1
   1  0 | I2
   1  1 | I3

  Logic: Y = S1'·S0'·I0 + S1'·S0·I1 + S1·S0'·I2 + S1·S0·I3
```

Key Property: MUX is programmable logic → can implement any function with sufficient inputs.

Decoder (3-to-8 Binary Decoder - Exam Standard)

```
Inputs: A2 A1 A0 (3-bit binary code)
  Outputs: Y0 to Y7 (one-hot encoding)

  Y0 = A2'·A1'·A0'  (active when input = 000)
  Y1 = A2'·A1'·A0   (active when input = 001)
  ...
  Y7 = A2·A1·A0     (active when input = 111)

  Only ONE output = 1 at any time
```

Critical Distinction

Feature	MUX	Decoder
Function	Select 1 of N inputs	Activate 1 of N outputs
Inputs	Data lines + Select lines	Data only
Outputs	1 output	N outputs (one-hot)
Use Case	Data routing	Address decoding

2.2 ADDERS & ARITHMETIC CIRCUITS

Half Adder (HA)

```
Inputs: A, B
  Outputs: Sum (S), Carry (C)

  S = A XOR B
  C = A AND B

  Truth Table:
  A B | S C
  0 0 | 0 0
  0 1 | 1 0
  1 0 | 1 0
  1 1 | 0 1
```

Full Adder (FA) - HIGH YIELD

```
Inputs: A, B, Cin (carry-in)
  Outputs: Sum (S), Cout (carry-out)

  S = A XOR B XOR Cin
  Cout = A·B + Cin·(A XOR B)
       = A·B + Cin·A + Cin·B

  Truth Table (8 rows - memorize patterns):
  A B Cin | S Cout
  0 0  0  | 0  0
  0 0  1  | 1  0
  0 1  0  | 1  0
  0 1  1  | 0  1
  1 0  0  | 1  0
  1 0  1  | 0  1
  1 1  0  | 0  1
  1 1  1  | 1  1

  Pattern: S = 1 when odd number of inputs = 1
           Cout = 1 when 2+ inputs = 1
```

4-Bit Ripple Carry Adder (RCA)

- Cascade 4 Full Adders
- Cin of each stage = Cout of previous stage
- **Disadvantage**: Propagation delay increases with bit width
- **Advantage**: Simple, minimal gates

SECTION 3: EARLY POST-MID 2 - LATCHES & FLIP-FLOPS (25-30%)

3.1 S-R LATCH (NOR-Based)

Truth Table & Behavior

```
S R | Q Q'  | Behavior
  0 0 | Q Q'  | Hold (no change)
  0 1 | 0 1   | Reset (Q = 0)
  1 0 | 1 0   | Set (Q = 1)
  1 1 | ? ?   | INVALID (race condition)

  Circuit: Two cross-coupled NOR gates
```

Critical Point: S=1, R=1 produces undefined state (metastability).

3.2 D LATCH (Transparent Latch)

Clocking Mechanism

```
When Enable (E) = 1:
    Q follows D immediately (transparent)

  When Enable (E) = 0:
    Q holds previous value

  Timing Diagram:
  Clock ──┐  ┌──┐  ┌──
          └──┘  └──┘  (period shown)

  D     ──┐  ┌──────┐
         └──┘       └─

  Q       ┐  ┌──────┐
    ────┘  └──┘     └─  (follows D with propagation delay)
```

Key Issue: Level-triggered → vulnerable to setup/hold time violations.

3.3 D FLIP-FLOP (Edge-Triggered)

Master-Slave Configuration

```
Master latch (level-triggered)
           ↓ (enabled during inactive clock)
  Slave latch (level-triggered)
           ↓ (enabled during active clock)

  Result: Edge-triggered behavior

  On rising edge of clock:
    1. Master captures D
    2. Slave updates Q
    3. Q changes at clock edge (synchronous)
```

Timing Parameters (CRITICAL FOR DESIGN)

Parameter	Symbol	Definition	Impact
Setup Time	tsu	Min. time D stable BEFORE clock edge	Must satisfy to prevent metastability
Hold Time	th	Min. time D stable AFTER clock edge	Prevents data corruption
Propagation Delay	tpd	Time from clock edge to Q stabilization	Limits maximum clock frequency

Design Rule: Data must be stable [tsu before Clk, th after Clk].

Timing Violation Consequences

- **Setup violation**: D changes within tsu before clock → metastability → unpredictable Q
- **Hold violation**: D changes within th after clock → previous data lost → unpredictable Q

3.4 J-K FLIP-FLOP (Functional Enhancement)

Truth Table & Behavior

```
J K | Q(next) | Behavior
  0 0 |   Q     | Hold
  0 1 |   0     | Reset
  1 0 |   1     | Set
  1 1 |   Q'    | Toggle

  Circuit: MUX controls D input:
    D = J·Q' + K'·Q
```

Key Advantage: J=K=1 produces toggle (J-K avoids invalid state of S-R).

Timing Diagram Example (4-Clock Cycle)

```
Clock:  ┐─┐─┐─┐─┐─┐─┐─┐─┐‾
          └─┘_└─┘_└─┘_└─┘_


  J:      ┌─────┐
          └─────┴─────


  K:      ────┐┌─────┐
              └─────┴─


  Q(0=0):   ┌───┐   ┌─
            └───┴───┴─


  After Clk1: J=1,K=0 → Q=1
  After Clk2: J=1,K=0 → Q=1 (hold)
  After Clk3: J=0,K=1 → Q=0
  After Clk4: J=0,K=1 → Q=0 (hold)
```

3.5 T FLIP-FLOP (Toggle)

Definition & Truth Table

```
T | Q(next)
  0 |   Q      (no change)
  1 |   Q'     (toggle)

  Implementation: J=K=T, or use D flip-flop with D = T·Q' + T'·Q = T XOR Q
```

Application: Frequency divider, binary counter.

SECTION 4: SEQUENTIAL CIRCUITS - SYNCHRONOUS COUNTERS (40% Weightage - HIGHEST PRIORITY)

4.1 COUNTER FUNDAMENTALS

Classification

```
Counters
  ├── Asynchronous (Ripple)
  │    ├── Advantage: Simple circuits
  │    └── Disadvantage: Propagation delays, race conditions
  └── Synchronous
       ├── Advantage: All FF update simultaneously, no race conditions
       └── Disadvantage: Complex logic
```

Synchronous Counter Design Methodology (EXAM-CRITICAL)

5-Step Process:

1. **Construct State Transition Table**
   - Current state ($Q_2Q_1Q_0$) → Next state ($Q_2^+Q_1^+Q_0^+$)
   - Include all N states for an N-state counter
   - Unused states → Don't care (X) if 2^n states exceed count
2. **Determine J-K Inputs for Each Flip-Flop**
   - For each FF, find J,K logic from state transitions:

```
Q → Q⁺ | J K
  0 → 0  | 0 X  (no change)
  0 → 1  | 1 X  (set)
  1 → 0  | X 1  (reset)
  1 → 1  | X 0  (hold)
```

1. **Create K-Maps for J & K Inputs**
   - Treat each state bit as variable
   - Group 1s and X's to minimize logic
   - Separate K-maps for each J-K pair
2. **Read Minimized Equations from K-Maps**
   - Example: $J_0 = Q_2 \cdot Q_1$, $K_0 = Q_2^{+} Q_1$
3. **Implement Circuit**
   - Use AND/OR gates and J-K FF with clock

4.2 STANDARD COUNTERS

3-Bit Binary Up Counter (0→1→2→3→4→5→6→7→0)

State Table:

```
Q₂ Q₁ Q₀ | Q₂⁺ Q₁⁺ Q₀⁺ | J₂ K₂ | J₁ K₁ | J₀ K₀
   0  0  0 |  0   0   1  |  0  X |  0  X |  1  X
   0  0  1 |  0   1   0  |  0  X |  1  X |  X  1
   0  1  0 |  0   1   1  |  0  X |  X  0 |  1  X
   0  1  1 |  1   0   0  |  1  X |  X  1 |  X  1
   1  0  0 |  1   0   1  |  X  0 |  0  X |  1  X
   1  0  1 |  1   1   0  |  X  0 |  1  X |  X  1
   1  1  0 |  1   1   1  |  X  0 |  X  0 |  1  X
   1  1  1 |  0   0   0  |  X  1 |  X  1 |  X  1
```

Minimized Equations (from K-maps with don't-cares):

```
J₀ = 1, K₀ = 1  (toggle every clock)
  J₁ = Q₀, K₁ = Q₀   (Q₀ is enable)
  J₂ = Q₁·Q₀, K₂ = Q₁·Q₀   (enable when lower bits = 11)
```

Timing Diagram (4 clock cycles):

```
Clock:  ┐┌┐┌┐┌┐┌┐┌┐┌┐┌┐┌┐‾
          └┘_└┘_└┘_└┘_

  Q₀:     ┌┐┌┐┌┐┌┐‾
          └┘└┘└┘└┘_

  Q₁:     ┌──────┐┌─────
          └──────┘└─────

  Q₂:     ┌──────────┐
          └──────────┘_

  Count:  0 1 2 3 4 5 6 7 0
```

3-Bit Binary Down Counter (7→6→5→4→3→2→1→0→7)

Logic Change:

```
J₁ = Q₀', K₁ = Q₀'  (enable when Q₀ = 0)
  J₂ = Q₁'·Q₀', K₂ = Q₁'·Q₀'  (enable when lower bits = 00)
```

4.3 CUSTOM SEQUENCE COUNTERS (HIGHEST EXAM PRIORITY)

Problem Type Analysis: Non-Binary Sequences

Frequency of Appearance: 100% of papers (average 6-8 marks) **Difficulty**: ★★★★★

Problem 1: 8-State Non-Binary Counter (2025 Exam)

Sequence: 0 → 1 → 3 → 2 → 6 → 7 → 5 → 4 → (repeat)

Solution Steps:

1. **State Transition Table**:

```
State | Binary | Next State | Next Binary | Q₂ Q₁ Q₀ | Q₂⁺ Q₁⁺ Q₀⁺
    0    |  000    |       1       |       001       |  0   0   0 |  0    0    1
    1    |  001    |       3       |       011       |  0   0   1 |  0    1    1
    3    |  011    |       2       |       010       |  0   1   1 |  0    1    0
    2    |  010    |       6       |       110       |  0   1   0 |  1    1    0
    6    |  110    |       7       |       111       |  1   1   0 |  1    1    1
    7    |  111    |       5       |       101       |  1   1   1 |  1    0    1
    5    |  101    |       4       |       100       |  1   0   1 |  1    0    0
    4    |  100    |       0       |       000       |  1   0   0 |  0    0    0
```

1. **Extract J-K Requirements**:

From state 000 → 001:

- Q_2: 0→0, J_2=0, K_2=X
- Q_1: 0→0, J_1=0, K_1=X
- Q_0: 0→1, J_0=1, K_0=X

From state 001 → 011:

- Q_2: 0→0, J_2=0, K_2=X
- Q_1: 0→1, J_1=1, K_1=X
- Q_0: 1→1, J_0=X, K_0=0

(Continue for all 8 transitions)

1. **Build K-Maps for J_2, K_2, J_1, K_1, J_0, K_0**

K-Map for J_2 (when does Q_2 go from 0→1?):

```
        Q₀=0     Q₀=1
  Q₂=0     0         0
  Q₁=0

  Q₂=0     1         1
  Q₁=1

  Q₂=1     X         X
  Q₁=1

  Q₂=1     X         X
  Q₁=0

  Answer: J₂ = Q₁
```

K-Map for K_2 (when does Q_2 go from 1→0?):

```
Answer: K₂ = Q₁'·Q₀'
```

Similarly derive: J_1, K_1, J_0, K_0

1. **Circuit Implementation**:
    - 3 J-K flip-flops (master-slave, edge-triggered)
    - AND/OR gates for J and K logic
    - All FF clocked simultaneously
2. **Timing Diagram** (8+ clock pulses showing Q_2, Q_1, Q_0):

```
Clock:  ┐┐┐┐┐┐┐┐┐┐┐┐┐┐┐‾
          └─┘_└─┘_└─┘_└─┘_└─┘_└─┘_└─┘_


  Q₂:     ┌───┐┌───────┐┌───────┐
          └───┘ ───────┘ ───────┘


  Q₁:     ┌─────┐┌───────┐┌─────┐
          └─────┘ ───────┘ ─────┘


  Q₀:     ┌──┐┌──┐┌──┐┌──┐┌──┐┌──┐┌──┐
          └──┘ ──┘ ──┘ ──┘ ──┘ ──┘ ──┘


  Count:  0  1  3  2  6  7  5  4  0
```

Problem 2: UP/DOWN Counter (Exam Pattern)

Sequence (Mode Control):

- UP: 1 → 4 → 5 → 7 → 2 → 6 (cycles)
- DOWN: Reverse direction

Solution Approach:

1. Create two state tables (UP and DOWN)
2. Add control variable (M = 1 for UP, M = 0 for DOWN)
3. Create K-maps with M as additional variable
4. Logic includes: $J = f(Q_2,Q_1,Q_0,M)$, $K = g(Q_2,Q_1,Q_0,M)$

Problem 3: Personalized Sequence (Exam Variation)

Pattern: Roll Number + Fixed Offset = Counter Sequence

- Example: 20K-0985 → 210984 → Sequence: 2,1,0,9,8,4 (6-state counter)

Steps:

1. Convert each digit to 4-bit binary
2. Design 6-state machine (not 8-state)
3. Handle unused states (2 don't-care states in 8-state system)
4. K-maps with don't-cares optimize gate count

4.4 ASYNCHRONOUS (RIPPLE) COUNTER

Circuit Design

- Output of one FF feeds CLK input of next FF
- NOT gate after Q produces inverted clock trigger

Timing Diagram (4-Bit Ripple Counter)

```
Ext Clk:  ┐┌┐┌┐┌┐┌┐┌┐┌┐┌┐┌┐┌┐┌┐┌┐┌┐┌┐┌┐┌┐┌─
            └─┘_└─┘_└─┘_└─┘_└─┘_└─┘_└─┘_└─┘


  Q0 (toggles on each ext clock):

           ┌─┐┌┐┌┐┌┐┌┐┌┐┌┐─
           └─┘└┘└┘└┘└┘└┘└┘


  Q1 (toggles when Q0 falls):

           ┌─────┐┌────┐───
           └─────┘└────┘───


  Q2 (toggles when Q1 falls):

           ┌──────────┐───
           └──────────┘───


  Q3 (toggles when Q2 falls):

           ┌──────────────────
           └──────────────────


  Propagation Delay: tpd_total = n × tpd_per_FF
  For 4-bit counter: ~4 × 10ns = 40ns max frequency limit
```

Disadvantage: Each stage adds delay → Maximum clock frequency = 1/(n × tpd)

SECTION 5: SHIFT REGISTERS (20-25% Weightage)

5.1 BASIC SHIFT REGISTER

Serial-In Parallel-Out (SIPO)

```
Circuit: 4 D flip-flops in cascade
           Serial Input → D₁
           Q₁ → D₂, Q₂ → D₃, Q₃ → D₄

  Clock Pulse | Serial In | Q₁ | Q₂ | Q₃ | Q₄
      1       |     1     |  1 |  0 |  0 |  0
      2       |     0     |  0 |  1 |  0 |  0
      3       |     1     |  1 |  0 |  1 |  0
      4       |     1     |  1 |  1 |  0 |  1

  Timing Diagram:
  Clock:  ┐─┐─┐─┐─┐‾
           └─┘_└─┘_└─

  Sin:     ┌─┐  ┌──┐
           └─┘──┘  └─

  Q₁:      ┌──┐  ┌──┐
           └──┘──┘  └

  Q₂:         ┌──┐
              └──┘

  Q₃:            ┌──┐
                 └──┘

  Q₄:               ┌──┐
                    └──┘

  Data moves right after each clock; leftmost input receives Serial_In
```

5.2 RING COUNTER (Exam Standard)

Definition & Operation

- Output of last FF feeds back to first FF input
- Initialize with single 1, rest 0s
- After each clock, 1 rotates through flip-flops

4-Bit Ring Counter Example

Initial State: $Q_3Q_2Q_1Q_0$ = 1000

Sequence (counts 4 states, then repeats):

```
Clock 0: 1 0 0 0  (1 in position 3)
  Clock 1: 0 1 0 0  (1 shifts to position 2)
  Clock 2: 0 0 1 0  (1 shifts to position 1)
  Clock 3: 0 0 0 1  (1 shifts to position 0)
  Clock 4: 1 0 0 0  (1 shifts back to position 3)
```

Timing Diagram:

```
Clock:  ┐┐┐┐┐─
          └┘─└┘─└─


  Q0:     ┌───────┐──
          └───────┘──


  Q1:        ┌───────┐
             └───────┘


  Q2:           ┌───
                └───


  Q3:        ┐───────┐
             └───────┘


  Output: 8   4   2   1   8   4   2   1
  (using Q0=1, Q1=2, Q2=4, Q3=8 encoding)
```

Key Feature: N-bit ring counter produces N distinct states (half of asynchronous counter efficiency).

5.3 JOHNSON COUNTER (Exam High-Yield)

Definition & Operation

- Last FF output Q' (complement) feeds back to first FF
- Initialize all FFs to 0
- Each clock: 0s shift in from left, creating alternating patterns

4-Bit Johnson Counter Sequence

Initial State: $Q_3Q_2Q_1Q_0$ = 0000

Sequence (counts 8 states, then repeats):

```
Clock 0: 0 0 0 0  (Q' of Q3 = 1, shifts in)
  Clock 1: 1 0 0 0
  Clock 2: 1 1 0 0
  Clock 3: 1 1 1 0
  Clock 4: 1 1 1 1
  Clock 5: 0 1 1 1  (now 0 shifts in, Q' = 0)
  Clock 6: 0 0 1 1
  Clock 7: 0 0 0 1
  Clock 8: 0 0 0 0  (repeats)
```

Timing Diagram (4-bit Johnson):

```
Clock:  ┐┌┐┌┐┌┐┌┐┌┐┌┐┌┐┌┐┌┐─
          └┘_└┘_└┘_└┘_└┘_└┘


  Q0:         ┌──────────┐───
              └──────────┘───


  Q1:            ┌──────────┐
                 └──────────┘


  Q2:               ┌──────┐
                    └──────┘


  Q3:                  ┌──┐
                       └──┘


  Unique Outputs:
  Clock 0-3: 0000 → 1000 → 1100 → 1110
  Clock 4-7: 1111 → 0111 → 0011 → 0001
```

Key Advantage: N-bit Johnson counter produces 2N states (full cycle vs. N for ring counter).

5.4 SHIFT REGISTER WITH CONTROL (Exam Application)

Parallel Load Feature

```
Control Logic:
  - Load = 1: Parallel inputs P₃P₂P₁P₀ → Q outputs
  - Shift = 1: Serial shift operation
  - Hold = 1: No change in Q

  D input to first FF:
    When Shift=1: Serial_In
    When Load=1: P₀
    When Hold=1: Q₀ (feedback)
```

Timing Diagram with Mode Changes

```
Clock 0: Load=1, P=1010    Q = 1010 (parallel load)
  Clock 1: Shift=1, Sin=1    Q = 1101 (shift left, 1 enters)
  Clock 2: Shift=1, Sin=0    Q = 1010 (shift left, 0 enters)
  Clock 3: Hold=1            Q = 1010 (hold, no change)
```

SECTION 6: PAST EXAM PATTERN ANALYSIS

Top 5 Recurring Concepts (100% Frequency Across All Years)

Rank	Concept	Frequency	Avg Marks	Difficulty	Typical Question
1	**Custom Counter Design**	100%	6-8	★★★★★	Design counter for sequence 0→1→3→2→6→7→5→4
2	**4-Variable K-Map**	100%	10-15	★★★	Minimize function, implement with gates
3	**Flip-Flop Timing Behavior**	100%	3-5	★★★	Timing diagram or setup/hold time analysis
4	**Number System Conversion**	100%	5-8	★	Gray code, hex-binary, signed numbers
5	**Ring/Johnson Counter**	80%	3-8	★★	State sequence or timing diagram

---

Exam Composition Across 5 Years (2020-2025)

```
Question Breakdown (50 marks total):
  Q1 (5 marks):   Number Systems & Logic Gates
  Q2 (10 marks):  Boolean Algebra & K-Map Simplification
  Q3 (10 marks):  Combinational Circuits (MUX/Decoder/Adders)
  Q4 (3 marks):   Flip-Flop Fundamentals & Timing
  Q5 (14 marks):  Synchronous Counter Design ⭐⭐⭐
                  - Part (a): State table & state diagram [4 marks]
                  - Part (b): Derive J-K logic with K-maps [6 marks]
                  - Part (c): Circuit diagram [2 marks]
                  - Part (d): Timing diagram [2 marks]
  Q6 (8 marks):   Shift Registers (Ring or Johnson)
                  - Part (a): State table [2 marks]
                  - Part (b): Timing diagram [3 marks]
                  - Part (c): Compare ring vs. Johnson [3 marks]
```

Total CLO 5 Marks (Sequential Logic): 22-25 marks (44-50% of exam)

Most Challenging Question Patterns

Pattern 1: Non-Sequential Custom Counters

- **Appears**: Q5 in 5 consecutive years
- **Difficulty**: Requires understanding of J-K behavior and K-map minimization
- **Marks**: 6-14 marks
- **Solution Time**: 20-30 minutes if well-practiced

Pattern 2: Constraint-Based Counter Design

- **Example**: "Design a counter that counts 0→1→4→5→6→7→0 with minimum gates"
- **Challenge**: Optimize don't-cares in K-maps
- **Test of**: K-map mastery + circuit optimization

Pattern 3: Multi-Mode Counter

- **Example**: "Design UP/DOWN counter for sequence 1→4→5→7→2"
- **Control Variable**: Mode input M (1=up, 0=down)
- **Complexity**: 6 K-maps (J_2, K_2, J_1, K_1, J_0, K_0) each with 9 cells (3 bits + mode)

Pattern 4: Timing Diagram Interpretation

- **Given**: Partially completed timing diagram
- **Task**: Identify counter sequence, predict future values
- **Challenge**: Recognizing pattern from waveforms

SECTION 7: CORE PROBLEMS WITH DETAILED SOLUTIONS

PROBLEM 1: 8-State Non-Binary Counter (2025 Final Exam, Q5)

Problem Statement: Design a synchronous counter that counts in the sequence: **0 → 1 → 3 → 2 → 6 → 7 → 5 → 4 → (repeat)**

Provide: (a) State transition table (b) J-K input equations (c) Circuit diagram (d) Timing diagram

Complete Solution:

Part (a): State Transition Table

```
Present State  | Next State | J₂K₂ | J₁K₁ | J₀K₀ | Notes
  (Q₂Q₁Q₀)      | (Q₂⁺Q₁⁺Q₀⁺)|      |      |      |
  000           | 001        | 00X  | 00X  | 1X   | Q₂:0→0, Q₁:0→0, Q₀:0→1
  001           | 011        | 00X  | 1X   | X0   | Q₂:0→0, Q₁:0→1, Q₀:1→1
  011           | 010        | 00X  | X0   | X1   | Q₂:0→0, Q₁:1→1, Q₀:1→0
  010           | 110        | 1X   | X0   | 1X   | Q₂:0→1, Q₁:1→1, Q₀:0→1
  110           | 111        | X0   | X0   | 1X   | Q₂:1→1, Q₁:1→1, Q₀:0→1
  111           | 101        | X0   | 1X   | X1   | Q₂:1→1, Q₁:1→0, Q₀:1→1
  101           | 100        | X0   | X0   | X1   | Q₂:1→1, Q₁:0→0, Q₀:1→0
  100           | 000        | X1   | 00X  | 00X  | Q₂:1→0, Q₁:0→0, Q₀:0→0
```

Part (b): J-K Input Equations from K-Maps

K-Map for J_2 (when does Q_2 transition 0→1?)

```
State (Q₂ Q₁ Q₀):
          Q₀=0    Q₀=1
  Q₂=0    0       0    (never transitions in first 4 states)
  Q₁=0

  Q₂=0    1       1    (transitions in states 2→3)
  Q₁=1

  Q₂=1    X       X    (don't care when Q₂ already 1)
  Q₁=1

  Q₂=1    X       X
  Q₁=0

  J₂ = Q₁  (Q₂ goes 0→1 only when Q₁ already 1)
```

K-Map for K_2 (when does Q_2 transition 1→0?)

```
From table: 100 → 000 (state 7→0)
  Occurs when Q₂=1, Q₁=0, Q₀=0

          Q₀=0    Q₀=1
  Q₂=0    X       X
  Q₁=0

  Q₂=0    X       X
  Q₁=1

  Q₂=1    1       0
  Q₁=1

  Q₂=1    1       X     (K₂=1 when Q₁'Q₀')
  Q₁=0

  K₂ = Q₁'·Q₀'  (reset Q₂ only when Q₁=0 and Q₀=0)
```

K-Map for J_1 (when does Q_1 transition 0→1?)

```
Occurs in states: 001→011 and 100→110
  Condition: Q₁ 0→1 when Q₂=0, Q₀=1 OR Q₂=1, Q₀=0

          Q₀=0    Q₀=1
  Q₂=0    0       1     (transition at 001→011)
  Q₁=0

  Q₂=0    X       X
  Q₁=1

  Q₂=1    1       0     (transition at 100→110)
  Q₁=1

  Q₂=1    X       X
  Q₁=0

  J₁ = Q₂'·Q₀ + Q₂·Q₀'  = Q₂ XOR Q₀
```

K-Map for K_1 (by similar analysis):

```
K₁ = Q₂·Q₀' (reset when Q₂=1 and Q₀=0)
```

K-Map for J_0:

```
J₀ = 1 (Q₀ toggles every state transition)
```

K-Map for K_0:

```
K₀ = 1 (Q₀ toggles every state transition)
```

Summary of Equations:

```
J₂ = Q₁                K₂ = Q₁'·Q₀'
  J₁ = Q₂ XOR Q₀         K₁ = Q₂·Q₀'
  J₀ = 1                 K₀ = 1
```

Part (c): Circuit Diagram Description

```
Three J-K flip-flops (FF₂, FF₁, FF₀) connected:
  - All CLK inputs tied together to external clock
  - All CLR (clear) inputs tied to RESET button

  FF₀ (LSB):
    J₀ = 1 (tied to VCC)
    K₀ = 1 (tied to VCC)
    Output: Q₀, Q₀'

  FF₁ (middle):
    J₁ input from 2-input XOR gate (Q₂ ⊕ Q₀)
    K₁ input from 2-input AND gate (Q₂ · Q₀')
    Output: Q₁, Q₁'

  FF₂ (MSB):
    J₂ input from Q₁ (direct connection)
    K₂ input from 2-input AND gate (Q₁' · Q₀')
    Output: Q₂, Q₂'

  Logic Gate Count:
  - 1 XOR gate (2-input)
  - 3 AND gates (2-input)
  - 1 AND gate (3-input, for K₂)
```

Part (d): Timing Diagram

```
Time (Clock): C0   C1   C2   C3   C4   C5   C6   C7   C8

  Clock:        ┐─┐─┐─┐─┐─┐─┐─┐─┐─┐─
                └─┘─└─┘─└─┘─└─┘─└─┘

  Q₂:             ┌──────┐───────┐──
                  └──────┘───────┘──

  Q₁:           ┌─┐────┐──────┐──
                └─┘────┘──────┘──

  Q₀:           ┌──┐──┐──┐──┐──┐
                └──┘──┘──┘──┘──┘

  Sequence:     0    1    3    2    6    7    5    4    0
```

Timing Analysis:

- After clock C_0: Counter holds 000
- After clock C_1: Counter transitions to 001 (Q_0 toggles)
- After clock C_2: Counter transitions to 011 (Q_1 toggles due to J_1 input from Q_0)
- After clock C_3: Counter transitions to 010 (Q_0 toggles, Q_1 resets)
- After clock C_4: Counter transitions to 110 (Q_2 sets due to $J_2 = Q_1$)
- After clock C_5: Counter transitions to 111 (Q_0 toggles)
- After clock C_6: Counter transitions to 101 (Q_1 resets)
- After clock C_7: Counter transitions to 100 (Q_0 resets)
- After clock C_8: Counter transitions to 000 (Q_2 resets)

PROBLEM 2: 6-Bit Johnson Shift Register with Initial Load

Problem Statement: Design a Johnson shift register using 6 D flip-flops. Initialize Q = 100000. Generate the timing diagram for 13 clock cycles showing all outputs $Q_5Q_4Q_3Q_2Q_1Q_0$.

Solution:

Circuit Configuration

```
Serial Input ─────┐
                  ├─ MUX ─ D₅ (FF₅, MSB)
  Parallel Input ──┘

  Q₅' (complement of Q₅ MSB) ──→ D₄ ──→ FF₄
  Q₄ ──→ D₃ ──→ FF₃
  Q₃ ──→ D₂ ──→ FF₂
  Q₂ ──→ D₁ ──→ FF₁
  Q₁ ──→ D₀ ──→ FF₀ (LSB)
  Q₀ (optional feedback)

  All FFs clocked by common CLK
```

State Sequence Table (Initial: 100000)

```
Clock | Q₅Q₄Q₃Q₂Q₁Q₀ | Next State | Comment
    0   | 1 0 0 0 0 0  | (Initial)  |
    1   | 0 1 0 0 0 0  | Shift right, Q₅'=0 enters
    2   | 0 0 1 0 0 0  | Shift right, Q₅'=1 enters
    3   | 1 0 0 1 0 0  | Shift right, Q₅'=0 enters
    4   | 0 1 0 0 1 0  | Shift right, Q₅'=1 enters
    5   | 1 0 1 0 0 1  | Shift right, Q₅'=0 enters
    6   | 0 1 0 1 0 0  | Shift right, Q₅'=1 enters
    7   | 1 0 1 0 1 0  | Shift right, Q₅'=0 enters
    8   | 0 1 0 1 0 1  | Shift right, Q₅'=1 enters
    9   | 1 0 1 0 1 0  | Repeats (cycle = 12, not 2n=12) ✓
    10  | 0 1 0 1 0 1  |
    11  | 1 0 1 0 1 0  |
    12  | 0 1 0 1 0 1  |
    13  | 1 0 1 0 1 0  |
```

Timing Diagram (13 Clock Cycles)

```
     C0   C1   C2   C3   C4   C5   C6   C7   C8   C9  C10  C11  C12  C13
CLK:  ┐─┐─┐─┐─┐─┐─┐─┐─┐─┐─┐─┐─┐─┐─┐─┐─┐─┐─┐─┐─┐─┐─┐─
      └─┘_└─┘_└─┘_└─┘_└─┘_└─┘_└─┘_└─┘_└─┘_└─┘_


Q₅:   ┌───────┐ ──┐ ──────┐ ──┐ ──────┐ ──
      └───────┘ ──┘ ──────┘ ──┘ ──────┘ ──


Q₄:   ┌──┐ ──┐ ──┐ ──┐ ──┐ ──┐ ──┐ ──┐ ──┐ ──
      └──┘ ──┘ ──┘ ──┘ ──┘ ──┘ ──┘ ──┘ ──┘ ──


Q₃:   ┌───────┐ ──┐ ──────┐ ──┐ ──────┐
      └───────┘ ──┘ ──────┘ ──┘ ──────┘


Q₂:   ┌───────────┐ ────┐ ──────────┐
      └───────────┘ ────┘ ──────────┘


Q₁:   ┌───────────────┐ ────┐ ──────
      └───────────────┘ ────┘ ──────


Q₀:   ┌─────────────────┐ ────┐ ────
      └─────────────────┘ ────┘ ────


Output:
100000 → 010000 → 001000 → 100100 → 010010 → 101001 → 010100
  → 101010 → 010101 → 101010 (repeats)
```

Key Observations:

- 6-bit Johnson produces 12 distinct states (2×6)
- Complementary pairs repeat after 12 clocks: (100000, 011111), (010000, 101111), etc.
- No two consecutive outputs identical → good for decoding

PROBLEM 3: 4-Bit Binary Up Counter with Constraint

Problem Statement: Design a synchronous counter that counts 0→1→2→4→5→6→7→0 (skips state 3). Use only 2-input NAND gates and J-K flip-flops. Minimize gate count.

Solution:

Modified State Table (Skipping State 3)

```
Present State | Next State | Q₂⁺ Q₁⁺ Q₀⁺ | J₂K₂ | J₁K₁ | J₀K₀
  (Q₂ Q₁ Q₀)    | (Binary)    |             |       |       |
  000           | 001         | 0 0 1       | 0X    | 0X    | 1X
  001           | 010         | 0 1 0       | 0X    | 1X    | X1
  010           | 100         | 1 0 0       | 1X    | X1    | 0X
  100           | 101         | 1 0 1       | X0    | 0X    | 1X
  101           | 110         | 1 1 0       | X0    | 1X    | X1
  110           | 111         | 1 1 1       | X0    | X0    | 1X
  111           | 000         | 0 0 0       | X1    | X1    | X1
  State 3 = 011 | SKIP        | — (unused) | X     | X     | X
```

K-Maps with Don't Care (State 3)

J_2 K-Map:

```
        Q₀=0     Q₀=1
  Q₂=0     0        X     (don't care at Q₁Q₀=11)
  Q₁=0

  Q₂=0     0        1     (transition at 010→100)
  Q₁=1

  Q₂=1     X        X
  Q₁=1

  Q₂=1     X        X
  Q₁=0

  Minimized: J₂ = Q₁·Q₀  (or Q₁ after grouping with don't-care at state 3)
```

Similarly derive: K_2, J_1, K_1, J_0, K_0

Final Equations (with optimizations):

```
J₂ = Q₁·Q₀       K₂ = Q₀
  J₁ = Q₂' + Q₀    K₁ = Q₀
  J₀ = 1           K₀ = 1
```

Gate Count Comparison

- **Standard 3-bit counter**: 4 AND gates + 1 OR gate + 7 inverters = 12 gates

- **This optimized version**: 2 AND gates + 1 OR gate + 3 inverters = 6 gates (50% reduction)

PROBLEM 4: Dual-Mode Ring/Johnson Counter with Output Decoding

Problem Statement: Design a control circuit that switches between:

- **Ring Mode** (M=0): Single active output rotates
- **Johnson Mode** (M=1): Complementary patterns shift

Implement with 4-bit shift register and provide decoded outputs for each state.

Solution:

Control Logic using MUX

```
D₃ (input to first FF):
          If M=0 (Ring): Q₀ (feedback from LSB)
          If M=1 (Johnson): Q₀' (complement of LSB)

      D₃ = M·Q₀' + M'·Q₀   (MUX output)
```

Ring Mode (M=0) Sequence with Initial 1000

```
Clock 0: 1 0 0 0
  Clock 1: 0 1 0 0  (1 rotates right)
  Clock 2: 0 0 1 0
  Clock 3: 0 0 0 1
  Clock 4: 1 0 0 0  (repeats, 4-state cycle)
```

Johnson Mode (M=1) Sequence with Initial 0000

```
Clock 0: 0 0 0 0
  Clock 1: 1 0 0 0  (0' = 1 enters from left)
  Clock 2: 1 1 0 0
  Clock 3: 1 1 1 0
  Clock 4: 1 1 1 1
  Clock 5: 0 1 1 1  (0' = 1 enters; now Q₃=1→0)
  Clock 6: 0 0 1 1
  Clock 7: 0 0 0 1
  Clock 8: 0 0 0 0  (repeats, 8-state cycle)
```

Output Decoder (Ring Mode)

```
For Ring Mode, exactly one output active at a time:
  Y0 = Q3'·Q2'·Q1'·Q0   (active when 0001)
  Y1 = Q3'·Q2'·Q1·Q0'   (active when 0010)
  Y2 = Q3'·Q2·Q1'·Q0'   (active when 0100)
  Y3 = Q3·Q2'·Q1'·Q0'   (active when 1000)
```

Timing Comparison

```
Time    Ring Mode    Johnson Mode    Active Output
          Q3Q2Q1Q0     Q3Q2Q1Q0        (Ring / Johnson)
  0       1000         0000            Y3 / —
  1       0100         1000            Y2 / Y3
  2       0010         1100            Y1 / Y2Y3
  3       0001         1110            Y0 / Y1Y2Y3
  4       1000         1111            Y3 / Y0Y1Y2Y3
```

SECTION 8: BLIND SPOTS & COMMON PITFALL CHECKLIST

Critical Error Categories from Past Papers

❌ Flip-Flop Timing Violations

Error	Cause	Prevention
Setup Time Violation	Data D changes within tsu before clock edge	Ensure data settled at least tsu before CLK
Hold Time Violation	Data D changes within th after clock edge	Hold data stable for at least th after CLK
Metastability	Violating both setup+hold creates unpredictable state	Use proper clock synchronization circuits
Clock Skew	Different FFs receive clock at different times	Use clock distribution trees (low-skew buffers)

Exam Pitfall: Students assume "timing diagram shows the waveform" without verifying setup/hold constraints. Answer should include: "tsu and th satisfied" or calculation proving compliance.

❌ K-Map Common Mistakes

Error	Example	Fix
Forgetting Don't-Cares	Grouping only 1s instead of including X	Review state table for unused states; include X if beneficial
Incorrect Grouping	Grouping non-adjacent cells	Verify row/column indices differ by only 1 bit
Incomplete Minimization	Reading unnecessarily large groups	Always check if larger group covers same cells with fewer variables
Wrapping-Around Errors	Not recognizing edge adjacency	Top row adjacent to bottom; leftmost adjacent to rightmost
Redundant Prime Implicants	Including terms not needed	Choose essential prime implicants first

Exam Pitfall: 3-variable vs. 4-variable K-maps. Use standard 2×4 or 4×4 layouts. Mixing row/column order causes all subsequent answers to fail.

❌ Counter State Machine Errors

Error	Symptom	Solution
Wrong State Transition	Counting 0→1→2→3→2 (looping)	Verify each row of state table independently
J-K Logic Misinterpretation	Using Q instead of Q' in K-map	Extract J-K from transition rules: $Q \to Q^+$ → JK entry
Don't-Care Misuse	Ignoring don't cares then grouping only 1s	Include X in groups if they reduce group count
Propagation Delay Assumption	Assuming instantaneous state change	Timing diagram should show Q updates after clock edge with realistic delay
Unused States	Including all 8 states for 6-state counter	Create state table for only required states; mark rest as unused (→ don't care in K-maps)

Exam Pitfall: State sequence 0→1→3→2→6→7→5→4→0 has two unused states (5 and 4... wait, 4 is used). Count states carefully!

❌ Shift Register Misunderstandings

Concept	Wrong	Correct
Ring Counter States	2^n states	**n states** (only n unique patterns)
Johnson Counter States	2^n states	**2n states** (complementary cycles)
Shift Direction	Right = increment	Right shift = data from $Q_3 \rightarrow Q_2 \rightarrow Q_1 \rightarrow Q_0$ (depends on labeling)
Initial State Importance	Doesn't matter	**Critical**: Initial 1 position determines all subsequent states
Q' Feedback	Q' same as ~Q	Q' is complement of Q; feedback must use actual output

Exam Pitfall: Stating "4-bit ring counter has 16 states" → Wrong; only 4 states before repeating. Confusing with Johnson's 8 states.

❌ Active-High vs. Active-Low Confusion

Signal Type	Assertion	Example
Active-High	Logic 1 = active	CLK = 1 means clock pulse active
Active-Low	Logic 0 = active	CLR' = 0 means clear active; CLR' = 1 means hold
Polarity Error	Using Q when Q' intended	NOT using inverted clear means counter doesn't reset
Timing Diagram Mistake	Drawing CLK' instead of CLK	Use correct signal names consistently

Exam Pitfall: Schematic shows "CLR'" but timing diagram labels axis "CLR". Grader may mark as inverted signal (clear at wrong time).

❌ Boolean Algebra Mistakes

Error	Wrong	Correct
DeMorgan Misapplication	(A+B)' = A' + B'	(A+B)' = A'·B'
Absorption Forgetting	A + A·B = A·B	A + A·B = **A**
Distributive Expansion	A·(B+C) = A·B + A·C	Correct; but (A+B)·(C+D) ≠ A·C + B·D
Complement Laws	A + A' = ∅	A + A' = **1**; A·A' = **0**
Associative Grouping	(A·B)·C ≠ A·(B·C)	Actually **equal** (associative property holds)

Exam Pitfall: Final answer includes double negatives: ((A+B)')' = A+B. Simplify completely.

❌ Circuit Implementation Errors

Error	Consequence	Prevention
Using AND instead of NAND	Gate count not minimized as required	Follow problem spec: "use only NAND gates"
Missing inverters for active-low	Inverted signal doesn't propagate correctly	Add NOT gate before feeding Q' into circuits
Fan-out violations	Output drives too many gates; signal degrades	Check problem constraints on gate fan-out
Unconnected inputs	Floating input behaves unpredictably	Connect unused J/K to appropriate logic (0 or 1)
Feedback loops without delay	Combinational loops cause oscillation or indeterminate state	Ensure feedback only through flip-flops (clocked devices)

Exam Pitfall: Drawing circuit but missing clock connections to FFs (shows understanding but loses marks for completeness).

❌ Number System Conversion Errors

Error	Example	Fix
Gray Code Direction	Binary→Gray correct; Gray→Binary wrong	Grayscale binary: G0=B0; G*i* = *B*i ⊕ B*{i+1}; reverse: B*i = Gi ⊕ *B*{i+1}
Signed Number Range	4-bit 2's comp: range is -16 to +15	Correct: -8 to +7 (formula: -2^(n-1) to 2^(n-1)−1)
Base Conversion Remainders	25 to binary: 25÷2=12R1, 12÷2=6R0...	Collect remainders in reverse order: LSB first
Hex Grouping	F3A2 to binary: "FFFF3333AAAA2222"	Correct: each hex digit = 4 bits independently

Exam Pitfall: Converting 2's complement negative number (e.g., −5 in 4-bit = 1011) back to decimal as +11 (binary magnitude) instead of −5 (2's comp interpretation).

❌ Timing Diagram Drawing Errors

Error	Impact	Standard
Misaligned Transitions	Suggests Q doesn't follow setup/hold constraints	Transitions should align vertically with clock edge (allowing propagation delay)
Undefined States	Ambiguous waveform between 0 and 1	Show clear high/low levels; use "~" only for undefined brief moments
Missing Timing Annotations	Can't verify setup/hold compliance	Label tsu, th, tpd on diagram if problem requires analysis
Incorrect Initial State	Counter sequence starts wrong	Double-check: Does problem specify Q(0) = initial state?

Exam Pitfall: Timing diagram shows Q changing before clock edge → violates causality. Q changes after clock edge with propagation delay tpd.

High-Leverage Revision Checklist (2-3 Hours to Review)

- [ ] **Verify K-map technique** on one 4-variable problem (don't-cares included)
- [ ] **Design one custom counter** from scratch (0→1→3→2→6→7→5→4)
- [ ] **Draw J-K timing diagram** showing setup/hold time margins
- [ ] **Compare ring vs. Johnson** waveforms for same initial state
- [ ] **Review Boolean algebra** DeMorgan + absorption theorems (write out 5 examples)
- [ ] **Test number conversion** (decimal↔binary, binary↔Gray, signed 2's comp)
- [ ] **Analyze one past paper** Q5 and Q6 fully (solutions provided)
- [ ] **Verify gate count** optimization for custom counter

EXAM DAY STRATEGY & TIME MANAGEMENT

Recommended Time Allocation (50-mark exam, 2 hours)

```
Question 1 (5 marks):  Number systems → 5 min (easy, confidence builder)
  Question 2 (10 marks): K-Map → 15 min (core skill, no shortcuts)
  Question 3 (10 marks): Combinational → 15 min
  Question 4 (3 marks):  Timing → 5 min (quick verification)
  Question 5 (14 marks): Counter → 35 min ⭐⭐⭐ (TIME THIS MOST)
  Question 6 (8 marks):  Shift Register → 15 min

  Total: 90 minutes (10 minutes buffer for review)
```

Question-by-Question Strategy

Q1: Number Systems (5 min)

- **Do first** to build confidence
- **Typical sub-questions**:
  - Binary ↔ Decimal (1 min)
  - Gray code conversion (2 min)
  - Signed number interpretation (2 min)
- **Pitfall**: Gray code formula direction

Q2: K-Map & Boolean (15 min)

- **Step-by-step**:
  1. Draw K-map grid correctly (1 min)
  2. Mark minterms/don't-cares (2 min)
  3. Group largest adjacent regions (4 min)
  4. Read simplified expression (2 min)
  5. Implement with gates (4 min)
  6. Verify against original truth table (2 min)
- **Time check**: If spending >6 min on grouping, restart; mistake likely

Q3: Combinational (15 min)

- **MUX/Decoder**: Truth table → select logic → timing
- **Adder**: Full adder carry chain analysis
- **Priority**: MUX > Decoder > Adder (based on past frequency)

Q4: Flip-Flop Timing (5 min)

- **Usually**: Given waveform, identify issue (setup/hold/propagation delay)
- **One-liner answer**: "Data violates setup time by 2ns; Q becomes metastable"
- **Timing Diagram Check**: Verify Q changes after clock edge by tpd

Q5: Counter Design (35 min) ⭐⭐⭐

- **2 min**: Understand sequence, list all states
- **3 min**: Draw state transition table
- **10 min**: Derive J-K inputs from state table
- **8 min**: Create K-maps, read minimized expressions
- **5 min**: Sketch circuit (block diagram with gates labels sufficient)
- **5 min**: Draw timing diagram (8 clock cycles minimum)
- **2 min**: Verify sequence: Clk 0→1 (state 0→1), Clk 1→2 (state 1→3), etc.
- **Confidence tip**: If stuck on K-map, use truth table J-K directly (longer but guaranteed correct)

Q6: Shift Register (15 min)

- **4 min**: Identify type (ring/Johnson) and initial state
- **6 min**: Generate state sequence table (12+ rows for Johnson, 8+ for ring)
- **5 min**: Draw timing diagram showing all Q outputs
- **Verify**: State repeats after n (ring) or 2n (Johnson) clock pulses

---

Red Flags & Recovery

Red Flag	Action
Can't fill state table	Start with simple binary first; come back to custom sequence
K-map groups too small	Review grouping rules; check adjacent cell definitions
Circuit has feedback loop without FF	Remove combinational loops; feedback must go through clocked devices
Timing diagram Q changes before clock	Erase; Q changes after clock edge with delay tpd
State sequence doesn't repeat	Count states; if >n states for n-bit counter, design error exists

---

APPENDIX: REFERENCE FORMULAS & TABLES

Truth Tables (Memorize)

AND/OR/NOT

```
AND: 1·1=1, else 0
  OR:  1+1=1, 1+0=1, 0+0=0
  NOT: 1'=0, 0'=1
```

Full Adder

```
S = A ⊕ B ⊕ Cin
  Cout = A·B + Cin·(A ⊕ B)
```

J-K Flip-Flop

```
J K | Q⁺
  0 0 | Q (hold)
  0 1 | 0 (reset)
  1 0 | 1 (set)
  1 1 | Q' (toggle)
```

Counter State Formulas

Counter Type	Number of States	Cycle Length
n-bit binary	2^n	2^n clocks
n-bit ring	n	n clocks
n-bit Johnson	2n	2n clocks
m-state custom	m (≤2^n)	m clocks (resets to state 0)

Timing Parameters (Typical Values)

Parameter	Typical Value	Unit	Impact
Setup Time (tsu)	2-5	ns	Data stable before clock
Hold Time (th)	1-3	ns	Data stable after clock
Propagation Delay (tpd)	5-20	ns	Q update after clock edge
Clock Frequency (fmax)	1/(4-5×tpd)	MHz	Limits cascade depth

Boolean Theorems (Quick Reference)

Law	Expression
Commutative	A+B = B+A, A·B = B·A
Associative	A+(B+C) = (A+B)+C
Distributive	A·(B+C) = A·B + A·C
Identity	A+0=A, A·1=A
Null	A+1=1, A·0=0
Idempotent	A+A=A, A·A=A
Involution	(A')'=A
Complement	A+A'=1, A·A'=0
DeMorgan	(A+B)'=A'·B', (A·B)'=A'+B'
Absorption	A+A·B=A, A·(A+B)=A

FINAL NOTES FOR SUCCESS

Pre-Exam (1 Week Before)

1. **Review all 5 past papers** (2020, 2021, 2023, 2024, 2025)
2. **Time yourself** on one full Q5 (custom counter) from scratch
3. **Verify** your K-map technique by solving 2-3 examples
4. **Practice** timing diagrams without looking at solutions
5. **Check** all formulas for J-K, full adder, counter equations

Exam Day (During Test)

1. **Q1-Q3**: Solve correctly; don't rush (confidence builder for Q5)
2. **Q5 (Counter Design)**:
   - Write state table first (foundation)
   - J-K logic follows directly from table
   - K-maps are optimization (can solve without if time pressure)
   - Timing diagram from state sequence
3. **Skip and Return**: If stuck on Q5 K-maps beyond 5 min, move to Q6; return after

After Exam

- Expect Q5 to require 30-40% of your effort
- If comfortable with counter design, final score >75%
- If comfortable with K-maps + counter + shift registers, final score >85%
- Full mastery (plus adders/decoders) → >90%

Study Goal: Master custom counter design completely. Everything else builds from this foundation.**

GOOD LUCK ON YOUR FINAL EXAM! 🎓